

Matematično-fizikalni praktikum: Diskretna Fourierova transformacija

Žan Ambrožič (28231062)

11. november 2025

1 Naloga

1. Izračunaj Fourierov obrat Gaussove porazdelitve in nekaj enostavnih vzorcev, npr. mešanic izbranih frekvenc. Za slednje primerjaj rezultate, ko je vzorec v intervalu periodičen (izbrane frekvence so mnogokratniki osnovne frekvence), z rezultati, ko vzorec ni periodičen (kako naredimo Gaussovo porazdelitev ‘periodično’ za FT?). Opazuj pojav potujitve na vzorcu, ki vsebuje frekvence nad Nyquistovo frekvenco. Napravi še obratno transformacijo (5) in preveri natančnost metode. Poglej, kaj se dogaja z časom računanja - kako je odvisen od števila vzorčenj?
2. Po Fourieru analiziraj 2,3s dolge zapise začetka Bachove Partite za violino solo, ki jih najdeš na spletni strani Matematično-fizikalnega praktikuma. Signal iz začetnih taktov Partite je bil vzorčen pri 44 100 Hz, 11 025 Hz, 5512 Hz, 2756 Hz, 1378 Hz in 882 Hz. S poslušanjem zapisov v formatu `.mp3` ugotovi, kaj se dogaja, ko se znižuje frekvenca vzorčenja, nato pa s Fourierovo analizo zapisov v formatu `.txt` to tudi prikaži.

2 Uvod

Pri numeričnem izračunavanju Fourierove transformacije

$$H(f) = \int_{-\infty}^{\infty} h(t) \exp\left(2\pi \iint ft\right) dt \quad (1)$$

$$h(t) = \int_{-\infty}^{\infty} H(f) \exp\left(-2\pi \iint ft\right) df \quad (2)$$

je funkcija $h(t)$ običajno predstavljena s tablico diskretnih vrednosti

$$h_k = h(t_k), \quad t_k = k\Delta, \quad k = 0, 1, 2, \dots, N-1. \quad (3)$$

Pravimo, da smo funkcijo vzorčili z vzorčno gostoto (frekvenco) $f = 1/\Delta$. Za tako definiran vzorec obstaja naravna meja frekvenčnega spektra, ki se imenuje *Nyquistova frekvenca*, $f_c = 1/(2\Delta)$: harmonični val s to frekvenco ima v vzorčni gostoti ravno dva vzorca v periodi. če ima funkcija $h(t)$ frekvenčni spekter omejen na interval $[-f_c, f_c]$, potem ji z vzorčenjem nismo odvzeli nič informacije, kadar pa se spekter razteza izven intervala, pride do *potujitve* (*aliasing*), ko se zunanji del spektra preslika v interval.

Frekvenčni spekter vzorčene funkcije (3) računamo samo v N točkah, če hočemo, da se ohrani količina informacije. Vpeljemo vsoto

$$H_n = \sum_{k=0}^{N-1} h_k \exp\left(2\pi \iint kn/N\right), \quad n = -\frac{N}{2}, \dots, \frac{N}{2}, \quad (4)$$

ki jo imenujemo diskretna Fourierova transformacija in je povezana s funkcijo v (1) takole:

$$H\left(\frac{n}{N\Delta}\right) \approx \Delta \cdot H_n.$$

Zaradi potujitve, po kateri je $H_{-n} = H_{N-n}$, lahko pustimo indeks n v enačbi (4) teči tudi od 0 do N . Spodnja polovica tako definiranega spektra ($1 \leq n \leq \frac{N}{2} - 1$) ustreza pozitivnim frekvencam $0 < f < f_c$, gornja polovica ($\frac{N}{2} + 1 \leq N - 1$) pa negativnim, $-f_c < f < 0$. Posebna vrednost pri $n = 0$ ustreza frekvenci nič (“istosmerna komponenta”), vrednost pri $n = N/2$ pa ustreza tako f_c kot $-f_c$.

Količine h in H so v splošnem kompleksne, simetrija v njih povzroči tudi simetrijo v drugih. Posebej zanimivi so trije primeri:

če je	h_k realna	tedaj je	$H_{N-n} = H_n^*$
	h_k realna in soda		H_n realna in soda
	h_k realna in liha		H_n imaginarna in liha

(ostalih ni težko izpeljati). V tesni zvezi s frekvenčnim spektrom je tudi moč. Parsevalova enačba pove, da je *Celotna moč* nekega signala neodvisna od reprezentacije

$$\sum_{k=0}^{N-1} |h_k|^2 = \frac{1}{N} \sum_{n=0}^{N-1} |H_n|^2.$$

Pogosto pa nas bolj zanima, koliko moči je vsebovane v frekvenčni komponenti med f in $f + df$, zato definiramo enostransko spektralno gostoto moči (*one-sided power spectral density*, PSD)

$$P_n = |H_n|^2 + |H_{N-n}|^2.$$

S takšno definicijo v isti koš mečemo negativne in pozitivne frekvence, vendar sta pri realnih signalih h_k prispevka enaka, tako da je $P_n = 2 |H_n|^2$.

Z obratno transformacijo lahko tudi rekonstruiramo h_k iz H_n

$$h_k = \frac{1}{N} \sum_{n=0}^{N-1} H_n \exp(-2\pi i \int_0^k n/N) \quad (5)$$

(razlika glede na enačbo (4) je le predznak v argumentu eksponenta in utež $1/N$).

3 Izračuni in rezultati

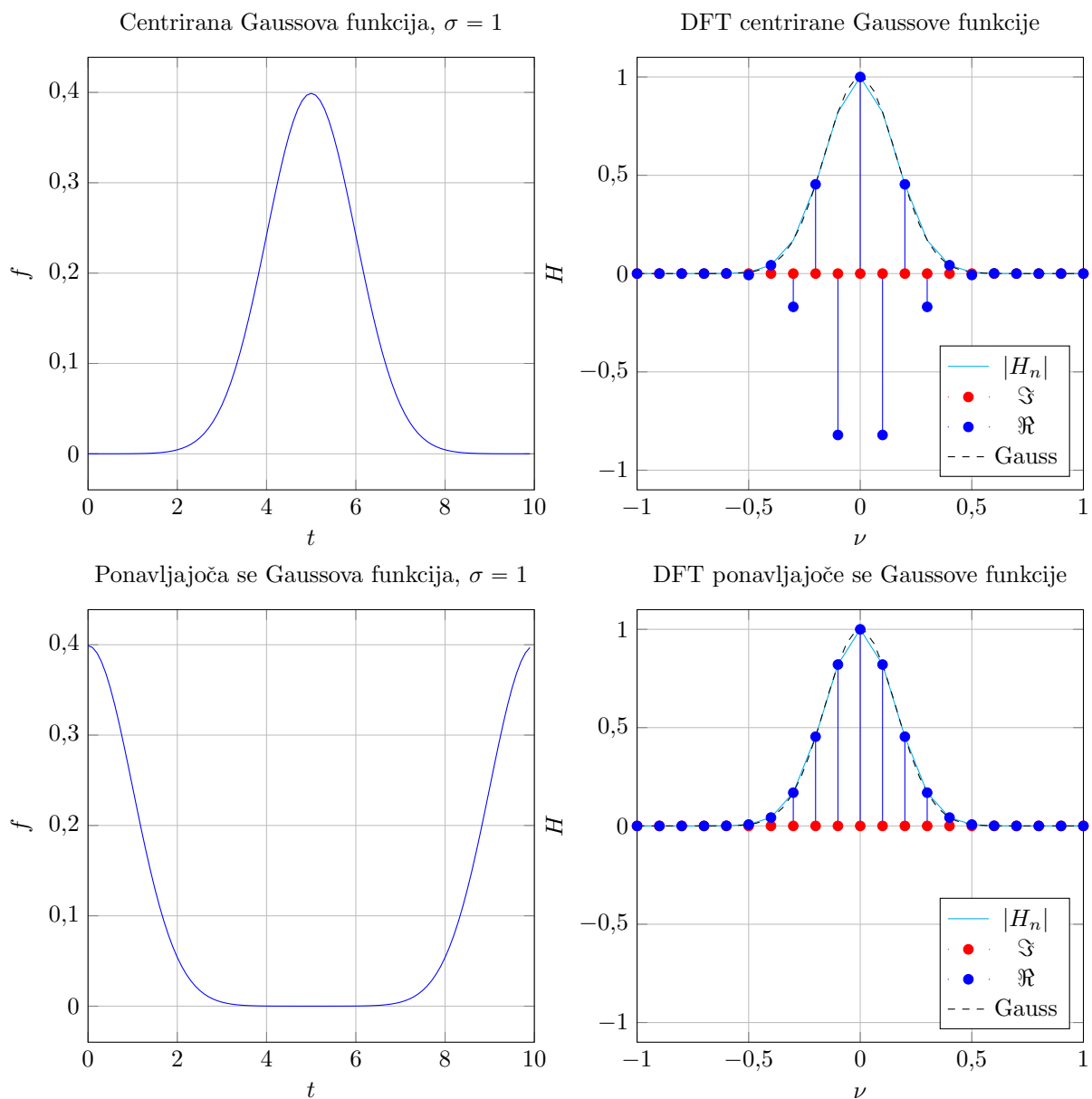
3.1 Gaussova porazdelitev

Najprej napravimo transformacijo Gaussove porazdelitve:

$$f(t) = \frac{e^{-t^2/2\sigma^2}}{\sqrt{2\pi\sigma^2}}, \quad (6)$$

ki jo lahko pustimo centrirano okoli sredine našega časovnega intervala ($t_0 = 5$ s), ali pa jo naredimo periodično tako, da vrh prestavimo na $t_0 = 0$, s čimer tudi na drugi strani $t = 10$ dobimo drugo stran funkcije. Teoretično bi morala biti transformacija Gaussove funkcije zopet Gaussova funkcija:

$$H(\nu) = e^{-\frac{(2\pi\nu\sigma)^2}{2}}. \quad (7)$$



Graf 1: Fourierovi transformaciji Gaussove funkcije s širino $\sigma = 1$ na intervalu $t \in [0, 10]$, vzorčenih je bilo 100 točk, frekvence na intervalu $\nu \in [-5, 5]$.

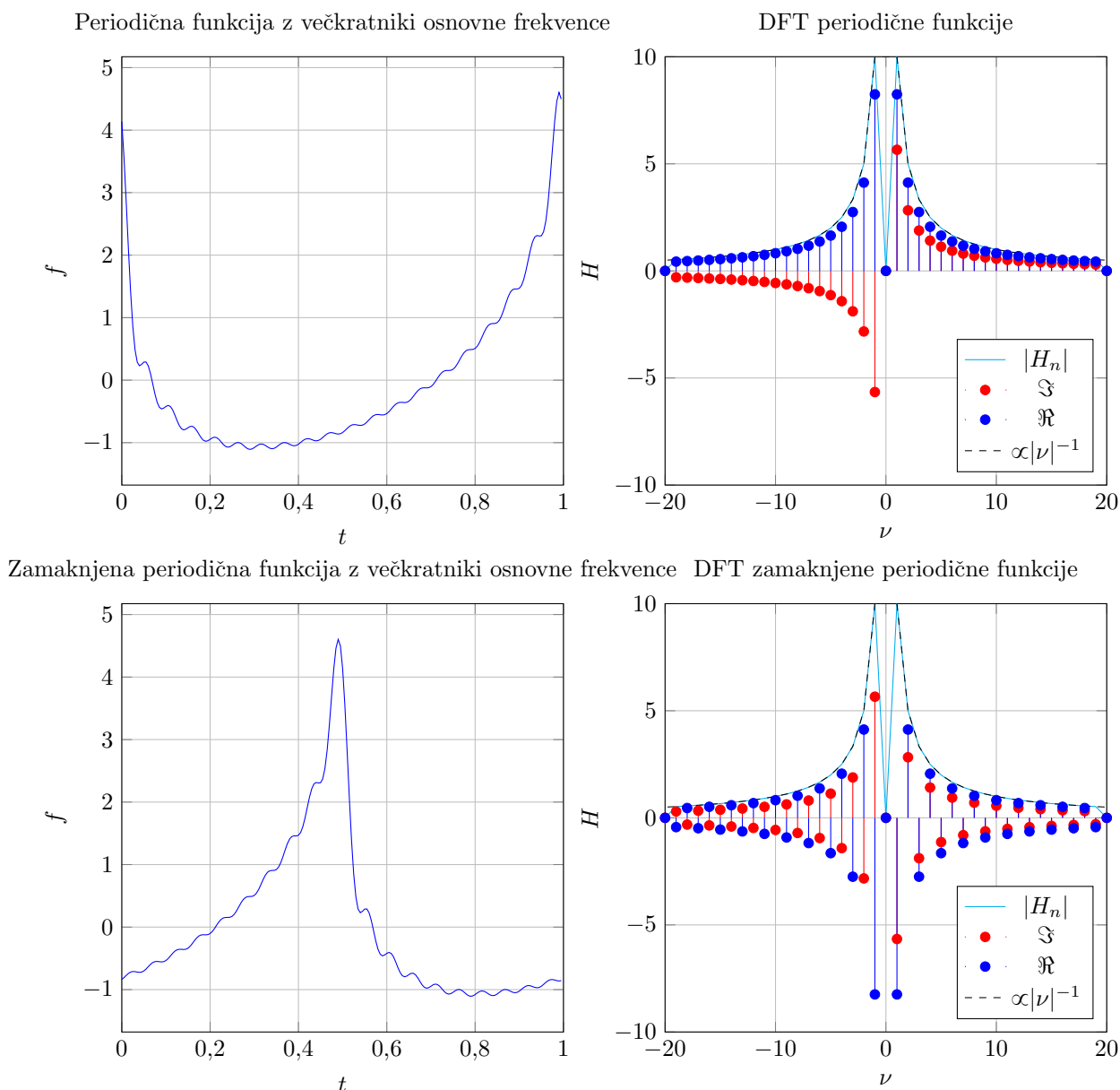
Opazimo, da je absolutna vrednost transformacije v obeh primerih enaka in v skladu s pričakovano Gaussovo porazdelitvijo transformacije, se pa za centrirano porazdelitev vsaka druga amplituda obrne (spremeni predznak).

3.2 Periodična funkcija

Za preizkus sestavimo testno periodično funkcijo z dovolj nizkimi frekvencami vsake od komponent:

$$f(t) = \sum_{k=1}^{20} \frac{1}{k} (1,166 \cos(2\pi kt) - 0,8 \sin(2\pi kt)) \quad (8)$$

S to obliko smo poskrbeli tudi, da absolutna vrednost pada kot ν^{-1} ter da bosta obe komponenti transformacije malo narazen.

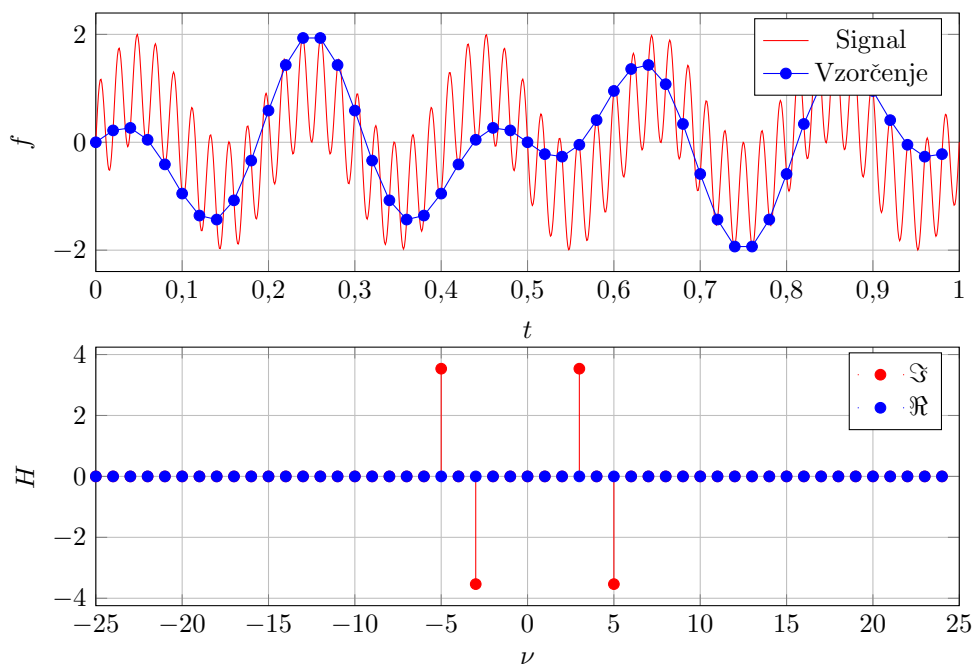


Graf 2: Fourierovi transformaciji periodične funkcije (8) na intervalu $t \in [0, 1]$ in z 200 točkami vzorčenja.

Po pričakovanjih se ovojnica (absolutne vrednosti) zopet dobro ujema s pričakovano obliko, spreminjajo se le predznaki komponent. Ker je testna funkcija realna, je realni del transformacije pričakovano sod, imaginarni del pa lih.

3.3 Potujitev

Če transformiramo signal s frekvencami nad Nyquistovo, opazimo potujitev – visoka frekvenca je izven območja frekvenc, ki jih dobimo s transformacijo, dobimo pa neničelne amplitude frekvenc, ki jih v resnici ni v signalu. Vzrok za to je očiten, če v signalu označimo vzorčne točke – takoj vidimo, da so vhodni podatki funkcije dejansko val z nižjo frekvenco in transformacija nam dejansko da pravilne rezultate za tak vzorec (programerji bi temu rekli SISO). Za tokratni testni vzorec vzamemo za referenco sinus s frekvenco 5 ter visokofrekvenčni sinus s frekvenco 1,88 Nyquistove, kar nam da signal, ki v eni časovni enoti, kolikor je dolg naš čas vzorčenja, zadošča ravno za tri periode, kar pri diskretnem vzorčenju natančno imitira signal s frekvenco 3.

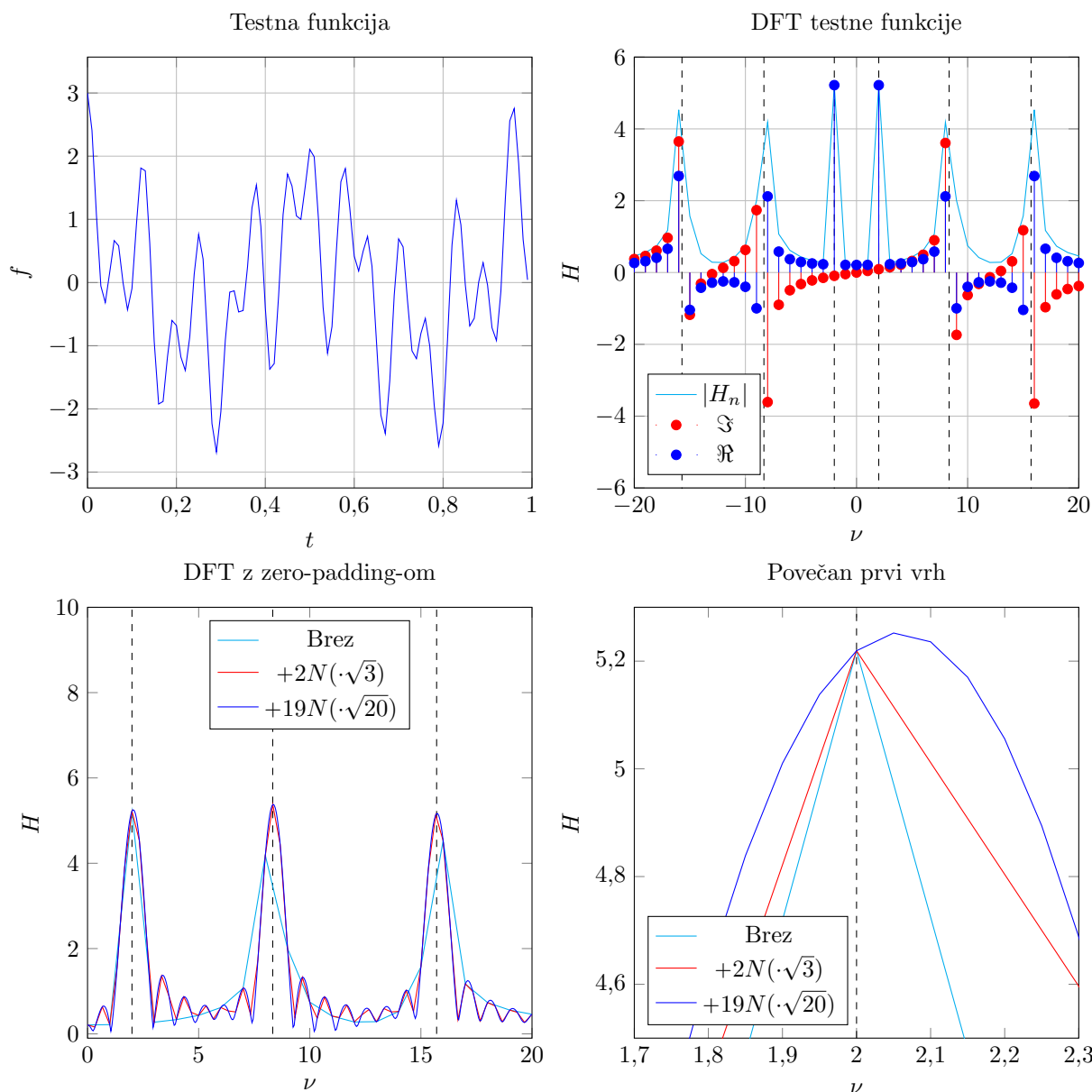


Graf 3: Prikaz potutitve visokofrekvenčne komponente signala pri Fourierovi transformaciji, 50 točk vzorčenja, Nyquistova frekvenca je 25.

Graf signala res nazorno prikazuje, da je vzorčeni signal res točno tak, kot sinusa s frekvencama 3 in 5, kar nam pove tudi Fourierova transformacija. Frekvenca je bila sicer namenoma izbrana tako, da je ima tudi vzorčen signal celoštevilsko frekvenco in dobimo oster vrh pri transformaciji.

3.4 Zero-padding

Če ima naš vhodni signal komponente, katerih frekvence niso celoštevilski večkratniki osnovne frekvence transformacije, vrhovi postanejo razmazani. Problem lahko naslovimo na dva načina: ali povečamo frekvenco vzorčenja, s čimer lahko povečamo frekvenčni razpon ali pa znižamo osnovno frekvenco (kar v praksi ni vedno mogoče), ali pa na konec signala dodamo ničle in s tem učinkovito povečamo število vzorčnih točk. Slednji pristop je vedno mogoč, ampak za razliko od prvega s seboj prinaša določene artefakte, ki se jih moramo zavedati. Za testni signal bomo tokrat vzeli celoštevilsko, racionalno in iracionalno frekvenco.



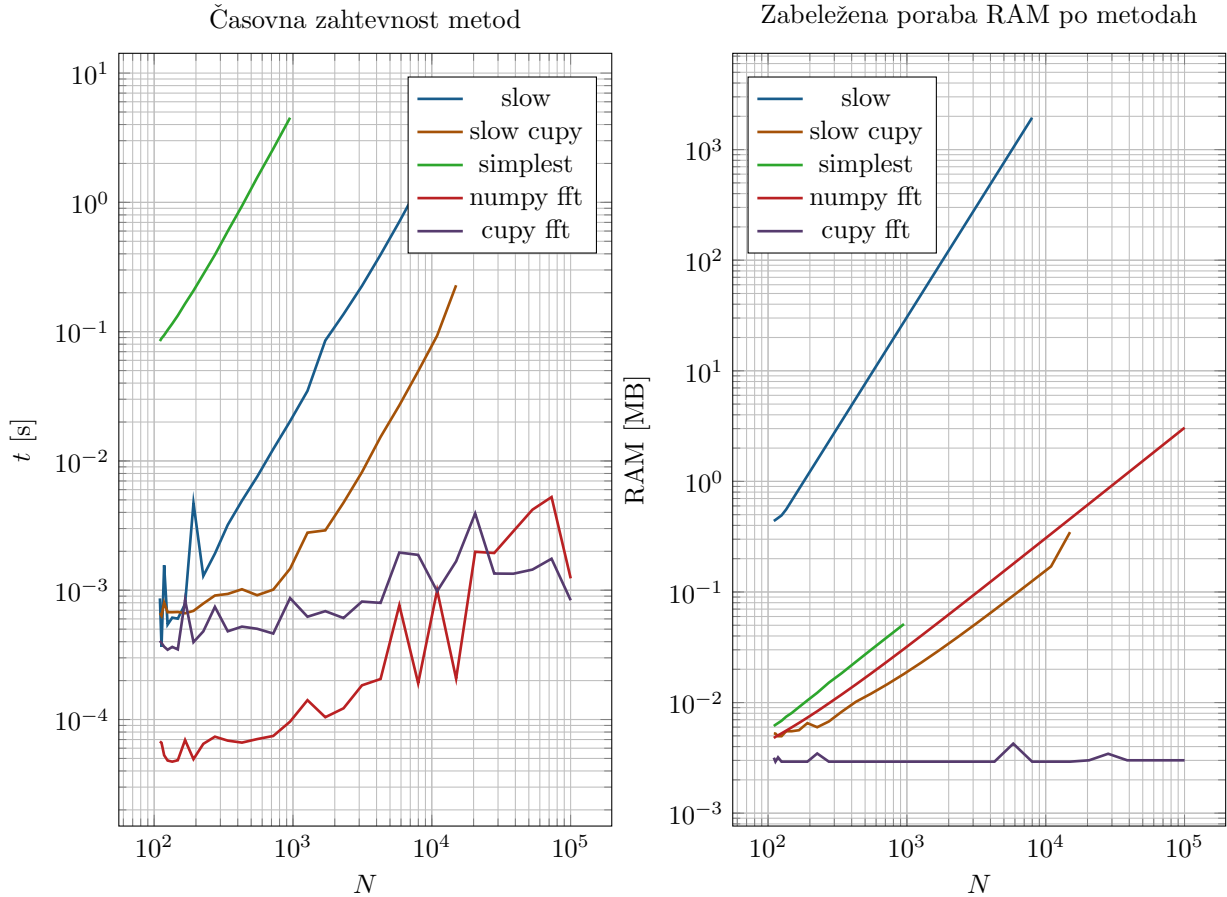
Graf 4: Fourierova transformacija sinusov s frekvencami 2, $25/3$ in $2,5\pi$, zero-padding z dvakratno in 19-kratno dolžino prvotnega vzorčenja.

Takoj opazimo dve značilnosti: vse diskretne točke transformacije z manjšim zero-padding-om se ujemajo s točkami višjih, poleg tega pa dobimo med glavnimi vrhovi še dodatne oscilacije (manjše vrhove, ki jih prej ni bilo). Zanimivo je, da se pri velikem zero-paddingu prvi vrh ($\nu = 2$) malo premakne od prave vrednosti, kar me je presenetilo – očitno prekomerna uporaba paddinga škodi in ga je treba uporabljati z občutkom, da dobimo čim boljše rezultate (poleg tega se podaljša tudi čas izračuna).

3.5 Časovna zahtevnost metod

Za izračun diskretne Fourierove transformacije poznamo več metod, primerjali bomo dve enostavni implementaciji s spletno učilnico (`DFT_simplest()`, ki direktno računa vsoto po uvodoma opisani metodi; `DFT_slow()`, ki uporablja vektorsko množenje namesto `for` zanke) in vgrajeno metodo `numpy.fft.fft()`. Slednji dve je smiselno preizkusiti še na grafični kartici, saj utegne biti matrično množenje v nekaterih primerih tam hitrejše. Uporabljamo računalnik z 32 GB RAM, procesorjem AMD Ryzen 7 5800x in grafično kartico Nvidia RTX 3060 z 12 GB spomina. Računali bomo transformacije iste testne funkcije, kot smo jo uporabljali za zero-padding, pri tem pa bomo spreminjali dolžino vzorca enostavno z dodajanjem ničel na koncu, kar seveda je pošten postopek, saj vrednosti praviloma ne vplivajo

na postopek transformacije. Dolžine vzorcev za teste bodo tako znašale med 110 in 10010 (s tem da bomo počasne metode predhodno nehali izvajati). Za pošteno primerjavo `cupy` z ostalimi metodami vsakič po končanem izračunu sprostimo spomin GPU, pred začetkom vseh testov pa poženemo obe `cupy` funkciji na testnih primerih, saj sicer dobimo lažne špice na prvi meritvi. Za beleženje spomina uporabljamo knjižnico `tracemalloc`, ki pa lahko beleži le RAM, zato nam ne pomaga prav dosti pri `cupy.fft.fft()`, ki teče v celoti na GPU. Za pošteno primerjavo zabeležimo spomin pred klicom funkcije, ki izračuna Fourierovo transformacijo, med izvajanjem pa zabeležimo vrh, rišemo pa razliko med vrhom in zasedenostjo pred začetkom.

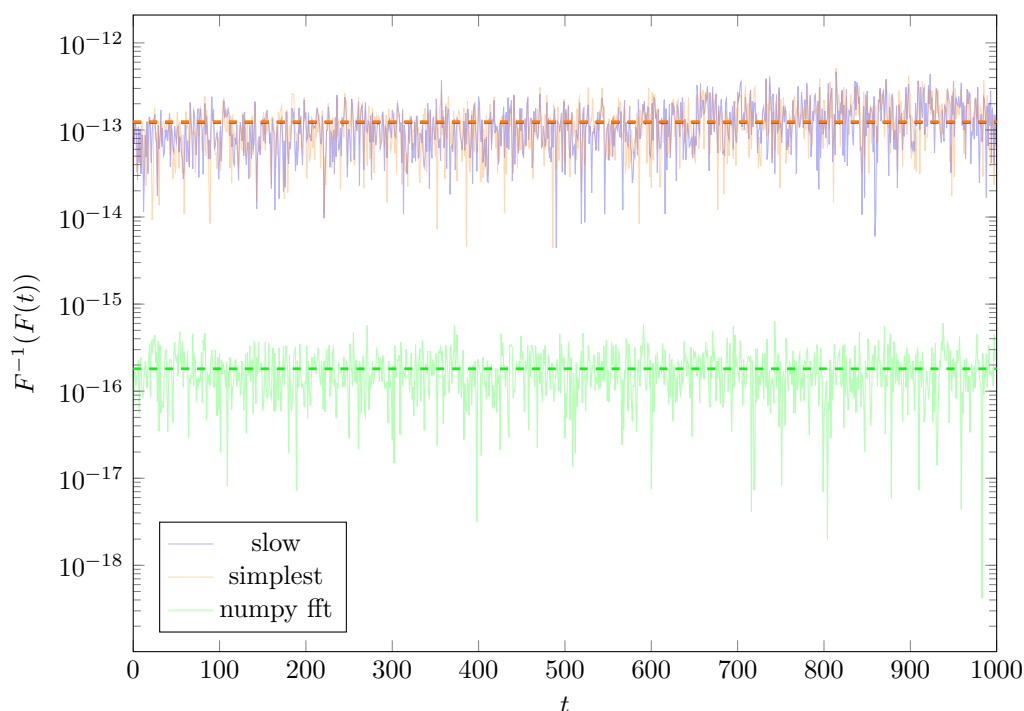


Graf 5: Časovna in prostorska zahtevnost metod za različno velike vzorce testnega signala z dodanimi ničlami na koncu.

Kot pričakovano, je najpočasnejša metoda `DFT_simplest()`, ki uporablja `for` zanke, znatno hitrejša sta izvedbi `DFT_slow()`, s tem da pri malo večjih vzorcih ($N > 1000$) veliko hitreje dela izvedba na GPU, se pa lepo vidi časovno zahtevnost $\mathcal{O}(N^2)$ pri vseh treh enostavno spisanih metodah. Pričakovano sta še boljši vgrajeni metodi, kjer GPU verzija prehití CPU verzijo šele pri zelo velikih vzorcih (meja je seveda odvisna od strojne opreme). Pri prostorski zahtevnosti pa se hitro pozna shranjevanje velikih matrik pri `DFT_slow()`, ki porabi največ pomnilnika (primerjava z GPU izvedbo nam pokaže, kolikšna je razlika med prostorsko zahtevnostjo shranjevanja podatkov in dejanskega izvajanja vmesnih korakov operacij, saj `DFT_slow` porablja spomin po N^2 za shranjevanje vseh matrik, medtem ko ga ostale izvedbe sorazmerno z N). Še enkrat velja opomniti, da smo beležili le RAM, zato GPU izvedbe (ki shranjujejo na notranji pomnilnik GPU) ne kažejo dejanske porabe pomnilnika.

3.6 Povratna transformacija

Prvoten signal lahko dobimo nazaj z inverzno Fourierovo transformacijo, ki v matematični teoriji da vrne eksakten signal. V praksi vemo, da na vsaki operaciji delamo s končno natančnostjo, kar nam prinese napako. Kako velika je, je odvisno od implementacije in števila decimalnih mest, s katerimi delamo. Za test tokrat ne bomo vzeli kakšne funkcije, ampak bomo poskusili rekonstruirati šum na za 1000 vzorčnih točk v intervalu $[-1, 1)$.



Graf 6: Napaka pri računanju inverzne transformacije transformacije signala.

Enostavno implementirani metodi imata natančnost okoli $(1,25 \pm 0,07) \cdot 10^{-13}$, medtem ko ima vgrajena metoda natančnost okoli $(1,8 \pm 1,0) \cdot 10^{-16}$, kar pa je že mejna natančnost 64-bitnega float-a. GPU-verzij nismo testirali, saj se razlikujejo le po glede na knjižnice `cupy` in `numpy`, ki pa naj bi dajali enake rezultate.

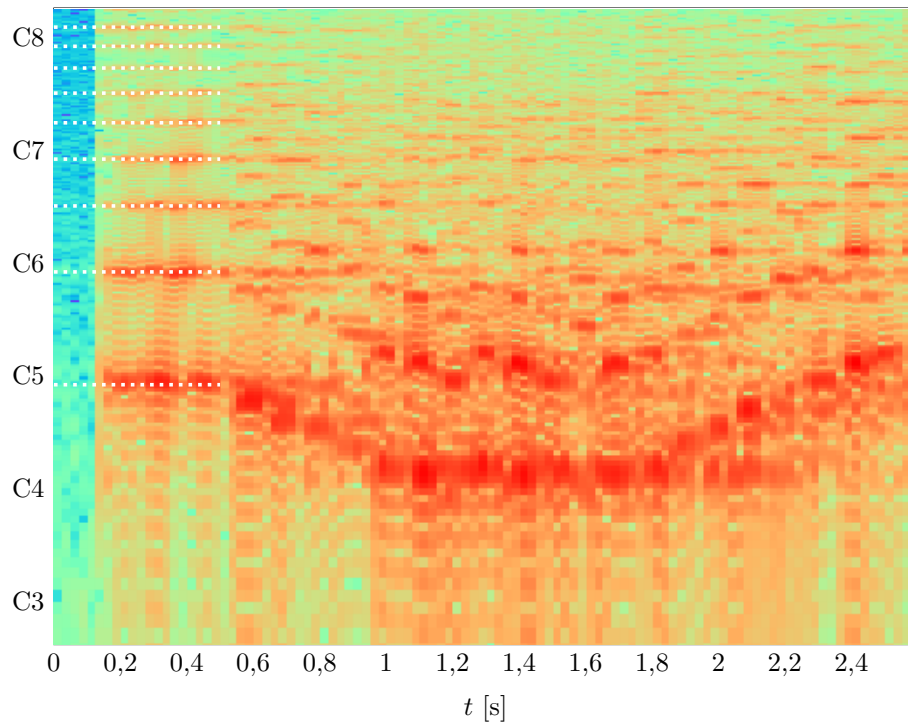
4 Analiza Bachove Partite

Za referenco poiščemo notni zapis Bachove Partite za violino št. 1, 4. stavek:

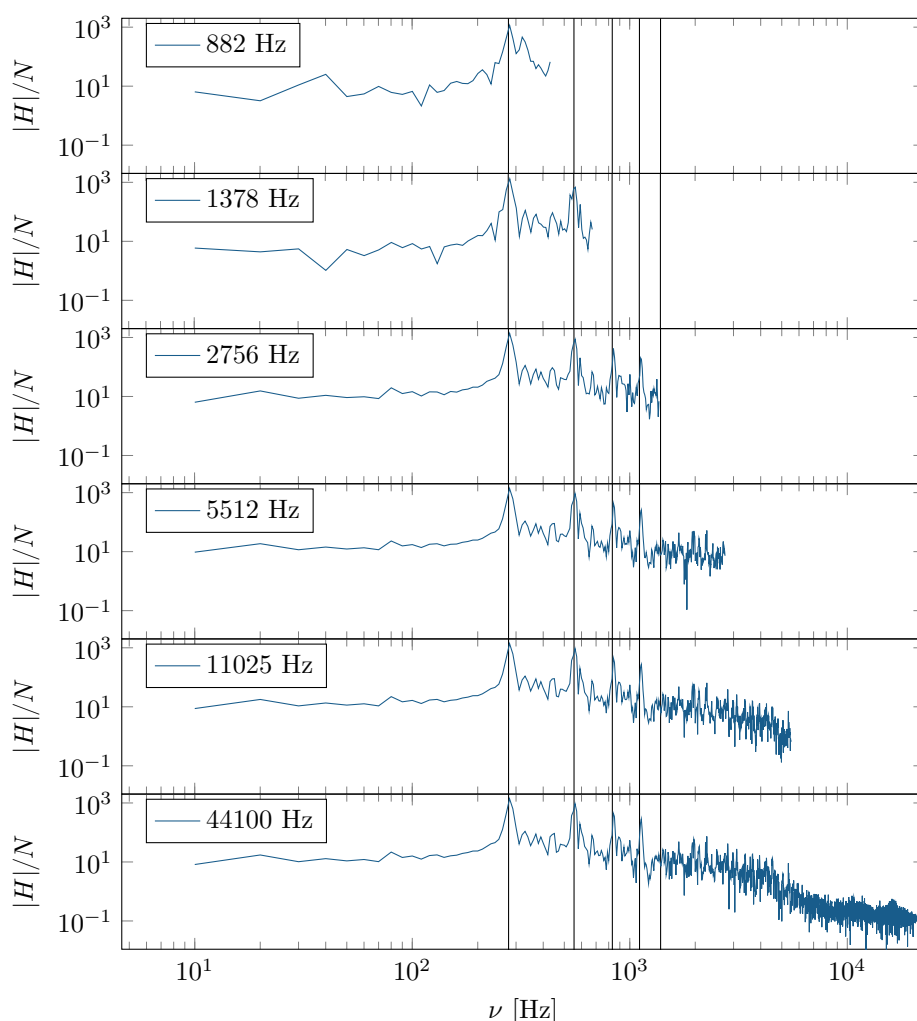


Slika 1: Prva dva takta Partite iz vzorca.

Najprej si oglejmo spektrogram. Uporabljali bomo vgrajeno funkcijo `numpy.fft.fft()`. Ker imamo omejeno količino podatkov za vsak zaigrani ton, moramo pametno izbrati okno. Vsaka šestnajstinka traja nekaj več kot desetinko sekunde, kar nam da okoli 4400 vzorčnih točk. Ker pa bomo težko zadeli vsako od not posebej ravno v pravem intervalu, se bolj splača vzeti nekoliko krajše intervale, npr. 4 na desetinko sekunde, s čimer si zagotovimo, da bo vsaj ena od merilnih točk v celoti ležala na enem tonu. Toni so določeni logaritemsko, oktavo, ki pomeni dvakratno frekvenco, sestavlja 12 poltonov, referenca pa je A4, ki ima frekvenco 440 Hz oz. v orkestrih pogosto tudi 442 Hz. Tako lahko umerimo našo frekvenčno skalo na tone, kar nam bo pomagalo primerjati rezultat z notnim zapisom. Za izboljšanje tonske ločljivosti uporabimo zero-padding in povečamo velikost vzorca na štirikratnik prvotne dolžine (to nam sicer da lažne nižje vrhove, ki pa so dovolj šibkejši od glavnih). Ker vemo, da je prvi in najdaljši ton v Partiti H4, ga lahko na začetku označimo, prav tako preverimo še alikvotne tone, ki so za violino (beri: struno) celoštevilski večkratniki osnovne frekvence.

Graf 7: Spektrogram odseka iz Partite z oknom $1/40$ s.

Na spektrogramu vidimo dobro ujemanje prvega tona in tudi oblike (kasnejših teoretičnih tonov nismo eksplicitno risali, nižji toni so tudi precej razmazani). Oglejmo si še primerjavo posnetkov z različnimi vzorčenji. Delamo Fourierovo transformacijo brez zero-paddinga na odseku posnetka od 1,052 s do 1,052 s.

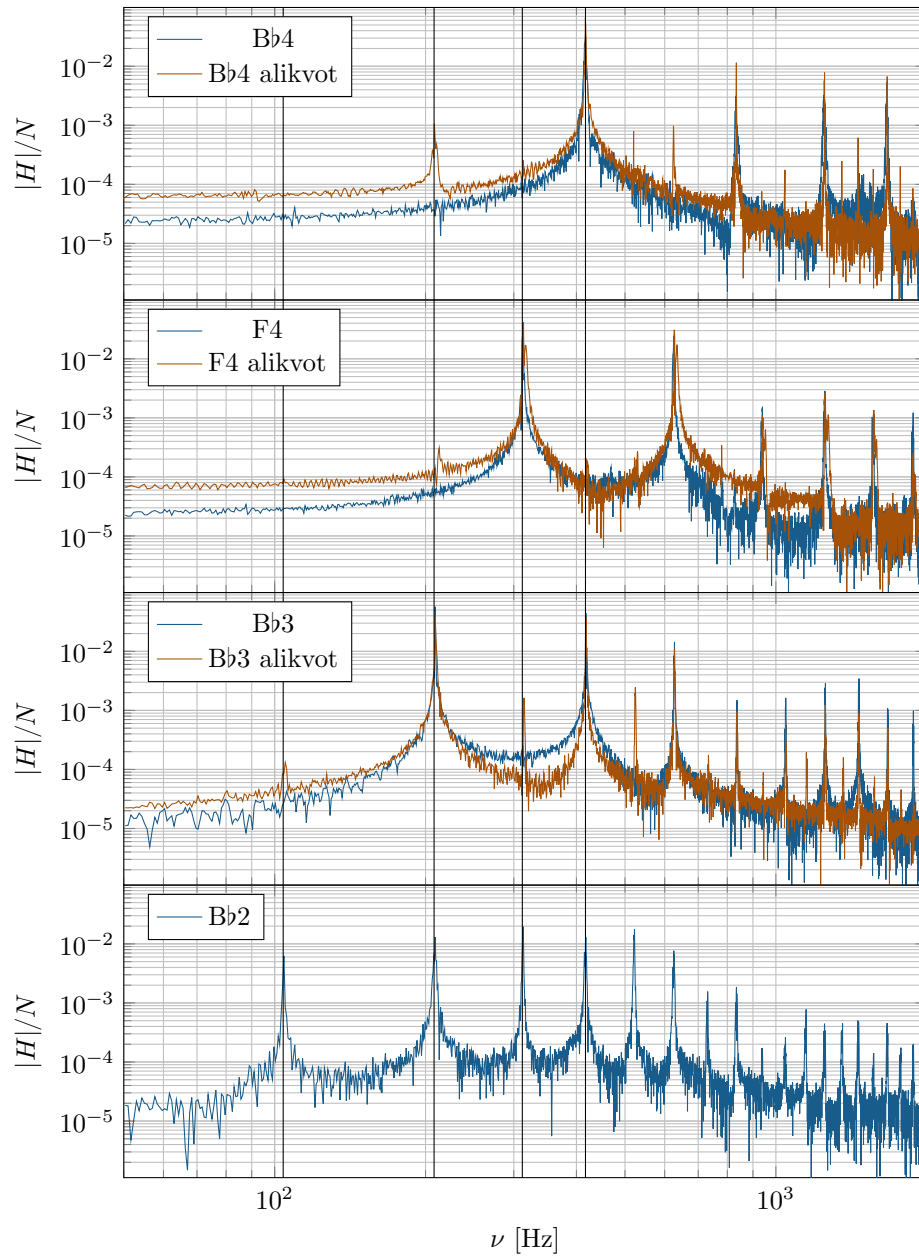


Graf 8: Transformacija desetinke sekunde posnetka z različnimi vzorčnimi hitrostmi.

Na grafu označimo še C $\sharp$ 2 (zadnja šestnajstinka v pred tretjo dobo) in njegove celoštevilske večkratnike, ki se dobro skladajo z ostalimi vrhovi (kar jih je vidnih pri omejenih vzorčenjih). V oknu verjetno ni izključno ta ton, zato so lahko na robovih primešane tudi druge frekvence, ampak je osnovni ton lepo viden, prav tako pa tudi prvih nekaj alikvotov. Viden je tudi razpon frekvenc, ki ga lahko opazujemo pri posameznem vzorčenju.

5 Alikvoti saksofona

V tem delu bomo raziskali še prvih nekaj alikvotnih tonov saksofona. Uglasitev naj bo na A4 = 442 Hz. Tenor saksofon je B $\flat$ instrument in lahko igra tone od spodnjega B $\flat$ (zveneči A $\flat$ 2), ki ima frekvenco okoli 104 Hz. Zaradi stožčastega profila cevi saksofoni delujejo kot na obeh straneh odprte cevi, alikvoti so tako celoštevilski večkratniki osnovne frekvence (intervalno gledano oktava, oktava in kvinta, dve oktavi ...). V našem poskusu bomo uporabljali celotno dolžino cevi (vse luknje zaprte) in gledali spektre tonov in alikvotov (s pravilnim pihanjem lahko ojačimo višje resonančne frekvence, koliko in katere je odvisno od sposobnosti in razporeditve saksofonista ter od pripravljenosti k sodelovanju jezička). Spektre pri višjih resonancah bomo nato primerjali s spektri tonov z odprtimi pravimi luknjami. Razlika je slišna, zanima pa nas, ali je dovolj velika, da jo lahko izmerimo z vgrajenim mikrofonom mobilne naprave. Spektri lepo pokažejo resonančne frekvence kot cele večkratnike osnovne frekvence pri posamezni konfiguraciji zaklopk (označene so frekvence vseh igranih tonov v B $\flat$ ključu: B $\flat$ 2, B $\flat$ 3, F4 in B $\flat$ 4). Kadar uporabljamo alikvot namesto pravega prijema, se v določeni meri primešajo še dodatne frekvence, ki jih sicer z uporabo pravega prijema ni moč opaziti (te so očitno razlog za malo drugačno barvo zvoka). Celotni odmiki amplitud so posledica različnih glasnosti igranja in manjših variacij oddaljenosti od mikrofona, bi bilo pa morda treba (predvsem pri nižjih frekvencah) upoštevati občutljivost mikrofona, vendar specifikacij nisem našel. Velja omeniti, da pri srednji in zgornji B $\flat$ (3 in 4)



Graf 9: Alikvoti najnižjega tona tenor saksofona in primerjava s pravilnimi prijemi za posamezen ton.

na saksofonih obstaja alternativni prijem, ki z drugim prijemom omogoča igranje istega tona, vendar je odprtina na enaki oddaljenosti od ustnika kot osnovna, zato ne pričakujemo velike razlike v spektru (se pa lahko določene druge zaklopke še vedno zapre pri uporabi alternativnega prijema, kar pa bi utegnilo malo vplivati na spekter). Razmerje višin vrhov je lahko odvisno tudi od načina igranja in jezička, zato velja upoštevati le položaje vrhov, sicer se tudi te da premikati (vse skupaj, ne le enega), saj saksofoni dovoljujejo, da s pravilno tehniko zvezno prehajamo med toni brez spremembe prijema (kar je odlična možnost za ustvarjanje raznih efektov, pa tudi za "fušanje"). Zanimivo bi bilo pogledati tudi spektra določenega tona s pravim prijemom in s pol tona višjim, ki ga nato znižamo s pomočjo ustne votline, ali tone nad osnovnim obsegom inštrumenta, ki jih lahko izsilimo na podoben način kot zgoraj opisane alikvote.

6 Zaključek

V nalogi smo raziskali osnovne lastnosti diskretne Fourierove transformacije, nekaj pasti, na katere moramo biti pozorni in nekaj trikov, kako lahko izboljšamo rezultate. Primerjali smo nekaj implementacij po časovni in prostorski zahtevnosti ter analizirali smo tudi realne zvočne posnetke.

Opomba: vsi grafi v tej nalogi so bili kot *proof-of-concept* iz Python-a pretvorjeni v L^AT_EX (tikz pgfplots) strojno brez vsakršnih ročnih popravkov s programom <https://github.com/ZanAmb/PltToTikz>.